Boss Bridge Protocol Audit Report

Version 1.1

Abdullah Amr

July 20, 2026

Prepared by: **Abdullah Amr**

Lead Auditor: **Abdullah Amr**

Table of Contents

Protocol Summary

Boss Bridge is a token-bridging mechanism designed to move an ERC20 token from Ethereum L1 to an L2 network under development. The reviewed codebase only contains the L1 contracts.

Users deposit tokens into an L1 vault by calling `L1BossBridge::depositTokensToL2`. A successful deposit emits a `Deposit` event that an off-chain service is expected to observe before minting the corresponding tokens on L2.

For L2-to-L1 withdrawals, an authorized bridge operator signs a withdrawal message. The user submits the signed message to `L1BossBridge`, which verifies the signer and executes the encoded L1 call.

The protocol includes the following security controls:

- The bridge owner can pause and unpause bridge operations.
- The owner can add or remove authorized signers.

- Deposits are subject to a global deposit cap.
- Withdrawals require an authorized operator signature.

The protocol relies heavily on the correctness and availability of its off-chain operators. A compromised signer, weak message validation, or inconsistent L2 accounting can place all funds held by the L1 vault at risk.

Disclaimer

This review was time-boxed and limited to the Solidity contracts listed in scope. Every reasonable effort was made to identify security issues; however, no audit can guarantee that all vulnerabilities have been discovered.

This report is not an endorsement of the protocol, its business model, or its operational security. Security also depends on deployment configuration, signer-key management, off-chain services, monitoring, and any L2 components that were not included in the reviewed codebase.

Risk Classification

Likelihood Impact	High	Medium	Low
High	High	High/Medium	Medium
Medium	High/Medium	Medium	Medium/Low
Low	Medium	Medium/Low	Low

Severity is assessed using the following general criteria:

- **High:** Direct loss of funds, creation of unbacked assets, permanent loss of core functionality, or compromise of a critical trust boundary.
- **Medium:** Significant disruption, gas griefing, or a security failure requiring particular assumptions or external conditions.
- **Low:** Limited operational, monitoring, compatibility, or maintainability impact.
- **Informational:** Best-practice, testing, or code-quality improvements without a direct exploitable security impact.

Audit Details

Audit Scope

The following contracts were included in the review:

```
src/L1BossBridge.sol
src/L1Token.sol
src/L1Vault.sol
src/TokenFactory.sol
```

Reviewed commit:

```
07af21653ab3e8a8362bf5f63eb058047f562375
```

Solidity version: `0.8.20`

Intended deployment targets:

- Ethereum Mainnet:
  - `L1BossBridge.sol`
  - `L1Token.sol`
  - `L1Vault.sol`
  - `TokenFactory.sol`
- ZKsync Era:
  - `TokenFactory.sol`

Token compatibility assumption:

Only `L1Token.sol` and exact behavioral copies of it are considered supported. Non-standard ERC20 behavior is outside the scope of this review.

Roles and Trust Assumptions

- **Bridge Owner:** Can pause or unpause the bridge and add or remove authorized signers. A compromised owner can alter the protocol's trusted signer set.
- **Signer / Operator:** Approves L2-to-L1 withdrawals by signing messages. A compromised or incorrectly implemented signer service can expose all vault funds.
- **User:** Deposits tokens on L1 and submits authorized withdrawal messages.
- **Vault:** Holds the L1 tokens backing the bridged L2 supply.
- **Off-chain Service:** Watches L1 deposit events, mints on L2, validates L2 withdrawals, and produces operator signatures.

Methodology

The review used:

- Manual source-code inspection
- Foundry unit testing
- Foundry fuzz testing
- Stateful invariant testing
- Static analysis with Slither and Aderyn
- Cross-contract trust-boundary analysis
- Signature and replay-protection analysis
- Review of deployment compatibility assumptions

Executive Summary

The reviewed implementation contains multiple critical trust and accounting failures. The most severe issues allow attackers to steal approved user tokens, mint unbacked L2 tokens, replay withdrawals, drain the vault through arbitrary signed calls, and block all deposits.

Several findings depend on the behavior of the off-chain signer service. These issues remain security-relevant because the on-chain contracts accept broadly scoped messages and provide little defense in depth against incorrect or compromised off-chain validation.

The original H-1 and H-2 issues have been consolidated because both arise from the same root cause: an attacker-controlled `from` parameter in `depositTokensToL2`. Both exploit variants are documented under H-1.

Issues Found

Severity	Number of Issues
High	7
Medium	1
Low	3
Informational	1
Total	**12**

Findings

High

[H-1] Attacker-controlled `from` parameter enables theft of approved tokens and infinite minting of unbacked L2 tokens

Description

`L1BossBridge::depositTokensToL2` allows the caller to provide an arbitrary `from` address:

```
function depositTokensToL2(
    address from,
    address l2Recipient,
    uint256 amount
) external whenNotPaused {
    if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
        revert L1BossBridge__DepositLimitReached();
    }

    token.transferFrom(from, address(vault), amount);
    emit Deposit(from, l2Recipient, amount);
}
```

The function does not require `from == msg.sender`.

This creates two exploit paths.

Exploit A: Stealing approved user tokens Any account that has approved the bridge can have its tokens transferred by an arbitrary caller. The attacker can set the victim as `from` and an attacker-controlled L2 address as `l2Recipient`.

The victim's tokens are moved into the vault, while the deposit event instructs the off-chain service to mint the corresponding L2 tokens to the attacker.

Exploit B: Infinite unbacked L2 minting During construction, the vault grants the bridge an unlimited token allowance:

```
vault.approveTo(address(this), type(uint256).max);
```

An attacker can therefore set both the token source and destination of the L1 transfer to the vault itself:

```
transferFrom(vault, vault, amount)
```

The vault's balance does not change, but the bridge emits a valid-looking `Deposit` event. This can be repeated indefinitely, causing the off-chain service to mint unbacked L2 tokens.

Impact

An attacker can:

- Steal the complete approved balance of any user who has approved the bridge.
- Mint an effectively unlimited quantity of unbacked tokens on L2.
- Break the bridge's collateralization invariant.
- Potentially redeem unbacked L2 tokens against legitimate assets held in the L1 vault.

Proof of Concept

Approved-token theft

```
function testAttackerCanDepositVictimApprovedTokens() public {
    uint256 victimBalance = token.balanceOf(user);

    vm.prank(user);
    token.approve(address(tokenBridge), type(uint256).max);

    address attacker = makeAddr("attacker");
    address attackerInL2 = makeAddr("attackerInL2");

    vm.prank(attacker);
    tokenBridge.depositTokensToL2(
        user,
        attackerInL2,
        victimBalance
    );

    assertEq(token.balanceOf(user), 0);
    assertEq(
        token.balanceOf(address(vault)),
        victimBalance
    );
}
```

Vault-to-vault deposits mint unbacked L2 tokens

```
function testVaultToVaultDepositsCanBeRepeated() public {
    address attackerInL2 = makeAddr("attackerInL2");
    uint256 vaultBalance = 100 ether;

    deal(address(token), address(vault), vaultBalance);

    for (uint256 i = 0; i < 10; ++i) {
```

```
        vm.expectEmit(address(tokenBridge));
        emit Deposit(
            address(vault),
            attackerInL2,
            vaultBalance
        );

        tokenBridge.depositTokensToL2(
            address(vault),
            attackerInL2,
            vaultBalance
        );

        assertEq(
            token.balanceOf(address(vault)),
            vaultBalance
        );
    }
}
```

The vault balance remains unchanged, but ten deposit events are emitted for the same collateral.

Recommended Mitigation

Remove the `from` parameter and always transfer tokens from `msg.sender`:

```
- function depositTokensToL2(
-     address from,
-     address l2Recipient,
-     uint256 amount
- ) external whenNotPaused {
+ function depositTokensToL2(
+     address l2Recipient,
+     uint256 amount
+ ) external whenNotPaused {
      if (totalDeposited + amount > DEPOSIT_LIMIT) {
          revert L1BossBridge__DepositLimitReached();
      }

-     token.transferFrom(from, address(vault), amount);
+     token.safeTransferFrom(
+         msg.sender,
+         address(vault),
+         amount
+     );

-     emit Deposit(from, l2Recipient, amount);
+     emit Deposit(msg.sender, l2Recipient, amount);
  }
```

Also reject zero-value deposits and ensure the off-chain minting service independently verifies that:

- The event was emitted by the canonical bridge.
- The transaction succeeded and reached sufficient finality.
- The actual vault balance increase matches the emitted amount.
- The deposit has not already been processed.

[H-2] Missing replay protection allows a single withdrawal signature to drain the vault

Description

`L1BossBridge::sendToL1` validates that a message was signed by an authorized signer, but it does not verify whether that message has already been executed.

```
function sendToL1(
    uint8 v,
    bytes32 r,
    bytes32 s,
    bytes memory message
) public whenNotPaused nonReentrant {
    address signer = ECDSA.recover(
        MessageHashUtils.toEthSignedMessageHash(
            keccak256(message)
        ),
        v,
        r,
        s
    );

    if (!signers[signer]) {
        revert L1BossBridge__Unauthorized();
    }

    (address target, uint256 value, bytes memory data) =
        abi.decode(message, (address, uint256, bytes));

    (bool success,) = target.call{value: value}(data);

    if (!success) {
        revert L1BossBridge__CallFailed();
    }
}
```

A signature authenticates the message but does not make it single-use. Because no nonce, withdrawal ID, or executed-message mapping exists, the same signature can be submitted repeatedly.

Impact

Any person who obtains one valid withdrawal signature can repeatedly execute it until the vault no longer has enough tokens.

The attacker does not need to compromise a signer key. Signature parameters become public when a legitimate transaction is broadcast.

Proof of Concept

```
function testWithdrawalSignatureCanBeReplayed() public {
    address attacker = makeAddr("attacker");

    uint256 vaultBalance = 1_000 ether;
    uint256 withdrawalAmount = 100 ether;

    deal(address(token), address(vault), vaultBalance);

    bytes memory message = abi.encode(
        address(token),
        0,
        abi.encodeCall(
            IERC20.transferFrom,
            (
                address(vault),
                attacker,
                withdrawalAmount
            )
        )
    );

    bytes32 digest =
        MessageHashUtils.toEthSignedMessageHash(
            keccak256(message)
        );

    (uint8 v, bytes32 r, bytes32 s) =
        vm.sign(operator.key, digest);

    for (uint256 i = 0; i < 10; ++i) {
        tokenBridge.withdrawTokensToL1(
            attacker,
            withdrawalAmount,
            v,
            r,
            s
        );
    }
```

```
39
40      assertEq(token.balanceOf(address(vault)), 0);
41      assertEq(token.balanceOf(attacker), vaultBalance);
42  }
```

A stateful invariant can express the broken property as:

```
function invariant_withdrawnCannotExceedAuthorized()
    public
    view
{
    assertLe(
        handler.totalWithdrawn(),
        handler.totalAuthorized(),
        "withdrawn amount exceeded signed authorization"
    );
}
```

A minimized failing sequence may be:

```
1. depositTokensToL2(...)
2. withdrawTokensToL1(...)
3. replayLastWithdrawal()
```

Recommended Mitigation

Add single-use withdrawal identifiers and mark them as consumed before the external call:

```
mapping(bytes32 withdrawalId => bool executed)
    public executedWithdrawals;

error L1BossBridge__WithdrawalAlreadyExecuted();

function sendToL1(
    uint8 v,
    bytes32 r,
    bytes32 s,
    bytes calldata message
) external whenNotPaused nonReentrant {
    bytes32 messageHash = keccak256(message);

    if (executedWithdrawals[messageHash]) {
        revert L1BossBridge__WithdrawalAlreadyExecuted();
    }

    address signer = ECDSA.recover(
        MessageHashUtils.toEthSignedMessageHash(
            messageHash
        ),
```

```
22          v,
23          r,
24          s
25      );
26
27      if (!signers[signer]) {
28          revert L1BossBridge__Unauthorized();
29      }
30
31      executedWithdrawals[messageHash] = true;
32
33      (address target, uint256 value, bytes memory data) =
34          abi.decode(message, (address, uint256, bytes));
35
36      (bool success,) = target.call{value: value}(data);
37
38      if (!success) {
39          revert L1BossBridge__CallFailed();
40      }
41  }
```

For stronger replay and domain protection, use EIP-712 typed data containing at least:

```
1  struct Withdrawal {
2      bytes32 withdrawalId;
3      address token;
4      address recipient;
5      uint256 amount;
6      uint256 nonce;
7      uint256 sourceChainId;
8      uint256 destinationChainId;
9      address verifyingBridge;
10 }
```

The consumed identifier must be unique for every L2 burn or withdrawal request.

[H-3] Broadly scoped signed messages allow arbitrary calls that can drain the vault

Description

`sendToL1` accepts a signer-approved message containing an arbitrary:

- `target`
- ETH `value`
- calldata payload

The contract does not restrict the target address, function selector, token, recipient, or amount.

Because `L1BossBridge` owns `L1Vault`, a signed message can call:

```
L1Vault.approveTo(attacker, type(uint256).max)
```

The attacker can then use `transferFrom` to drain all vault tokens.

A valid operator signature is still required. The vulnerability becomes exploitable when the signer service signs user-supplied bytes without strictly decoding and validating the complete message. This is particularly dangerous because the documented off-chain validation is not enforced on-chain.

Impact

A malicious or insufficiently validated withdrawal request can grant an attacker unlimited allowance over the vault and lead to complete loss of all bridged assets.

The arbitrary-call design also increases the blast radius of a compromised signer from authorizing incorrect withdrawals to executing any call that the bridge itself is permitted to make.

Proof of Concept

```
function testSignedArbitraryCallCanDrainVault() public {
    address attacker = makeAddr("attacker");
    uint256 vaultBalance = 1_000 ether;

    deal(address(token), address(vault), vaultBalance);

    // Models a signer service that only checks that the
    // requester previously submitted a bridge deposit.
    vm.prank(attacker);
    tokenBridge.depositTokensToL2(
        attacker,
        address(0),
        0
    );

    bytes memory message = abi.encode(
        address(vault),
        0,
        abi.encodeCall(
            L1Vault.approveTo,
            (attacker, type(uint256).max)
        )
    );

```

```
        bytes32 digest =
            MessageHashUtils.toEthSignedMessageHash(
                keccak256(message)
            );

        (uint8 v, bytes32 r, bytes32 s) =
            vm.sign(operator.key, digest);

        tokenBridge.sendToL1(v, r, s, message);

        token.transferFrom(
            address(vault),
            attacker,
            token.balanceOf(address(vault))
        );

        assertEq(token.balanceOf(address(vault)), 0);
        assertEq(token.balanceOf(attacker), vaultBalance);
    }
```

Recommended Mitigation

Do not expose a generic signer-authorized arbitrary-call primitive for ordinary token withdrawals.

Use a typed withdrawal function that constructs the only permitted token transfer internally:

```
function withdrawTokensToL1(
    Withdrawal calldata withdrawal,
    bytes calldata signature
) external nonReentrant whenNotPaused {
    _validateAndConsumeWithdrawal(
        withdrawal,
        signature
    );

    token.safeTransferFrom(
        address(vault),
        withdrawal.recipient,
        withdrawal.amount
    );
}
```

At minimum, enforce all of the following on-chain:

- `target == address(token)`
- `value == 0`
- Selector equals `IERC20.transferFrom.selector`
- Source address equals `address(vault)`

- Token equals the canonical bridge token
- Withdrawal ID has not been executed
- Signed recipient and amount exactly match the executed transfer

The signer service must independently decode and validate typed fields rather than sign opaque user-supplied bytes.

[H-4] Raw-bytecode assembly deployment is incompatible with the intended ZKsync Era deployment model

Description

`TokenFactory::deployToken` accepts arbitrary creation bytecode and deploys it with inline assembly:

```
function deployToken(
    string memory symbol,
    bytes memory contractBytecode
) public onlyOwner returns (address addr) {
    assembly {
        addr := create(
            0,
            add(contractBytecode, 0x20),
            mload(contractBytecode)
        )
    }

    s_tokenToAddress[symbol] = addr;
    emit TokenDeployed(symbol, addr);
}
```

The reviewed documentation states that `TokenFactory` will be deployed on ZKsync Era.

Native EraVM deployment requires the compiler and transaction to know the deployed contract bytecode hash in advance. A factory that accepts arbitrary runtime-supplied creation bytecode cannot satisfy this requirement using the Ethereum-style raw-bytecode `create` pattern.

As a result, the factory's core deployment path is incompatible with the intended native EraVM deployment environment.

Impact

`TokenFactory` may be undeployable or unable to deploy tokens on the intended ZKsync Era target.

This breaks a core protocol deployment requirement and can render the L2 token deployment process unusable.

Proof of Concept

The incompatibility can be reproduced by compiling and deploying the factory with native ZKsync tooling and then calling:

```
factory.deployToken(
    "BBT",
    type(L1Token).creationCode
);
```

The factory attempts to deploy arbitrary supplied bytecode through assembly. Native EraVM cannot safely transform this unrestricted pattern into the required `ContractDeployer` call with known bytecode dependencies.

Relevant ZKsync documentation:

- https://docs.zksync.io/zksync-protocol/era-vm/differences/evm-instructions
- https://docs.zksync.io/zksync-protocol/era-vm/differences/contract-deployment

Recommended Mitigation

Avoid accepting arbitrary creation bytecode. Deploy compiler-known contract types:

```
function deployToken(
    string calldata symbol,
    address recipient
) external onlyOwner returns (address addr) {
    L1Token deployedToken = new L1Token(recipient);
    addr = address(deployedToken);

    s_tokenToAddress[symbol] = addr;
    emit TokenDeployed(symbol, addr);
}
```

Alternatively, use ZKsync's supported deployment tooling and `ContractDeployer` system contract with properly supplied factory dependencies.

Deployment behavior should be tested on the exact ZKsync compiler, VM mode, and network configuration intended for production.

[H-5] Unsolicited token transfers can permanently consume the global deposit capacity

Description

The deposit limit is checked against the vault's raw ERC20 balance:

```
if (
    token.balanceOf(address(vault)) + amount
        > DEPOSIT_LIMIT
) {
    revert L1BossBridge__DepositLimitReached();
}
```

ERC20 transfers cannot prevent users from transferring tokens directly to the vault without calling the bridge.

An attacker can directly transfer `DEPOSIT_LIMIT` tokens to the vault. Because these tokens were not deposited through `depositTokensToL2`, no L2 tokens are minted for them, but the raw vault balance still reaches the cap.

Every subsequent legitimate deposit then reverts.

The original report's attack description was incorrect: an attacker cannot exceed the limit by using `depositTokensToL2`, because the check prevents that transaction. The actual bypass is a direct ERC20 transfer to the vault.

Impact

An attacker can block the bridge's L1-to-L2 deposit functionality for all users.

The bridge remains blocked until sufficient tokens leave the vault. An attacker can continue transferring tokens directly to the vault to keep the balance at the limit.

This creates a persistent denial of service against a core protocol function.

Proof of Concept

```
function testDirectTransferToVaultBlocksDeposits()
    public
{
    address attacker = makeAddr("attacker");
    uint256 limit = tokenBridge.DEPOSIT_LIMIT();

    deal(address(token), attacker, limit);

    vm.prank(attacker);
    token.transfer(address(vault), limit);

    deal(address(token), user, 1 ether);

    vm.startPrank(user);
    token.approve(address(tokenBridge), 1 ether);

    vm.expectRevert(
        L1BossBridge
            .L1BossBridge__DepositLimitReached
            .selector
    );

    tokenBridge.depositTokensToL2(
        user,
        user,
        1 ether
    );

    vm.stopPrank();
}
```

Recommended Mitigation

Do not use the vault's raw token balance as the deposit-cap accounting variable.

Track successful bridge deposits explicitly:

```
uint256 public totalOutstandingDeposits;

function depositTokensToL2(
    address l2Recipient,
    uint256 amount
) external whenNotPaused {
    if (amount == 0) {
        revert L1BossBridge__ZeroAmount();
    }

    uint256 updatedOutstanding =
        totalOutstandingDeposits + amount;
```

```
13
14      if (updatedOutstanding > DEPOSIT_LIMIT) {
15          revert L1BossBridge__DepositLimitReached();
16      }
17
18      totalOutstandingDeposits = updatedOutstanding;
19
20      token.safeTransferFrom(
21          msg.sender,
22          address(vault),
23          amount
24      );
25
26      emit Deposit(msg.sender, l2Recipient, amount);
27  }
```

Decrease the accounting value only after a valid, single-use L2 withdrawal has been authenticated and executed.

Direct token donations should not affect bridge capacity. Do not remove the deposit limit unless the protocol intentionally accepts unlimited exposure.

[H-6] Withdrawal authorization is not bound to an authenticated L2 burn or exact bridged amount

Description

`withdrawTokensToL1` does not verify an L2 burn, withdrawal proof, or bridge accounting record. It only verifies an operator signature over the requested L1 call.

```
1  function withdrawTokensToL1(
2      address to,
3      uint256 amount,
4      uint8 v,
5      bytes32 r,
6      bytes32 s
7  ) external {
8      sendToL1(
9          v,
10         r,
11         s,
12         abi.encode(
13             address(token),
14             0,
15             abi.encodeCall(
```

```
                IERC20.transferFrom,
                (address(vault), to, amount)
            )
        )
    );
}
```

This is not exploitable without a valid signer signature. However, if the off-chain service only checks that the requester previously made a successful deposit, an attacker can make a minimal deposit and request a withdrawal for a much larger amount.

The original PoC expected the oversized withdrawal to revert because the vault lacked funds. That does not prove the stated vulnerability. A correct PoC must give the vault sufficient funds and demonstrate that the oversized signed withdrawal succeeds.

Impact

Under insufficient off-chain validation, an attacker can withdraw substantially more than the amount they bridged or burned on L2, consuming funds belonging to other users and potentially draining the vault.

The on-chain contract cannot distinguish a legitimate withdrawal from an incorrectly signed one.

Proof of Concept

```
function testSmallDepositCanAuthorizeLargeWithdrawal()
    public
{
    address attacker = makeAddr("attacker");

    uint256 attackerDeposit = 1 ether;
    uint256 victimLiquidity = 1_000 ether;
    uint256 requestedWithdrawal = 1_000 ether;

    deal(
        address(token),
        address(vault),
        victimLiquidity
    );
    deal(address(token), attacker, attackerDeposit);

    vm.startPrank(attacker);
    token.approve(
        address(tokenBridge),
        attackerDeposit
    );
```

```
22
23      tokenBridge.depositTokensToL2(
24          attacker,
25          attacker,
26          attackerDeposit
27      );
28      vm.stopPrank();
29
30      // Models an operator that checks only whether the
31      // requester has previously made a deposit.
32      bytes memory message = abi.encode(
33          address(token),
34          0,
35          abi.encodeCall(
36              IERC20.transferFrom,
37              (
38                  address(vault),
39                  attacker,
40                  requestedWithdrawal
41              )
42          )
43      );
44
45      bytes32 digest =
46          MessageHashUtils.toEthSignedMessageHash(
47              keccak256(message)
48          );
49
50      (uint8 v, bytes32 r, bytes32 s) =
51          vm.sign(operator.key, digest);
52
53      tokenBridge.withdrawTokensToL1(
54          attacker,
55          requestedWithdrawal,
56          v,
57          r,
58          s
59      );
60
61      assertEq(
62          token.balanceOf(attacker),
63          requestedWithdrawal
64      );
65  }
```

Recommended Mitigation

A bridge withdrawal should be authorized by proof of an exact L2 state transition, normally an L2 burn or canonical withdrawal message.

The signed or proven payload should bind:

- Unique L2 withdrawal ID
- L2 transaction or message identifier
- Token address
- Exact burned amount
- Exact L1 recipient
- Source and destination chain IDs
- Verifying bridge address
- Nonce or message sequence number

Do not enforce that a user may only withdraw the amount they originally deposited on L1, because bridged tokens may legitimately be transferred between users on L2. Instead, verify that the exact withdrawal amount was burned or otherwise finalized on L2.

If a signer-based architecture is retained, the signer service must validate the exact L2 burn and must never sign an opaque user-provided call.

[H-7] `TokenFactory` permanently locks the initial token supply

Description

`L1Token` mints its entire initial supply to `msg.sender`:

```
1 constructor() ERC20("BossBridgeToken", "BBT") {
2     _mint(
3         msg.sender,
4         INITIAL_SUPPLY * 10 ** decimals()
5     );
6 }
```

When deployed through `TokenFactory`, `msg.sender` inside the token constructor is the factory contract, not the factory owner or intended token recipient.

`TokenFactory` has no function that can transfer ERC20 tokens it holds. Therefore, the full initial supply remains locked in the factory.

Impact

The complete initial supply of every `L1Token` deployed through the factory can become permanently inaccessible.

The tokens cannot be distributed to users, transferred to the bridge vault, or used for protocol operations. This can make the token deployment unusable.

Proof of Concept

```
function testFactoryLocksInitialTokenSupply()
    public
{
    TokenFactory factory = new TokenFactory();

    address tokenAddress = factory.deployToken(
        "BBT",
        type(L1Token).creationCode
    );

    L1Token deployedToken = L1Token(tokenAddress);

    uint256 initialSupply =
        1_000_000 * 10 ** deployedToken.decimals();

    assertEq(
        deployedToken.balanceOf(address(factory)),
        initialSupply
    );

    assertEq(
        deployedToken.balanceOf(address(this)),
        0
    );
}
```

Recommended Mitigation

Pass an explicit recipient to the token constructor:

```
contract L1Token is ERC20 {
    uint256 private constant INITIAL_SUPPLY =
        1_000_000;

    constructor(address recipient)
        ERC20("BossBridgeToken", "BBT")
    {
        if (recipient == address(0)) {
            revert L1Token__InvalidRecipient();
        }

        _mint(
```

```
            recipient,
            INITIAL_SUPPLY * 10 ** decimals()
        );
    }
}
```

For an Ethereum-style bytecode factory:

```
error TokenFactory__DeploymentFailed();

function deployToken(
    string calldata symbol,
    bytes memory contractBytecode,
    address recipient
) external onlyOwner returns (address addr) {
    bytes memory creationCode = abi.encodePacked(
        contractBytecode,
        abi.encode(recipient)
    );

    assembly {
        addr := create(
            0,
            add(creationCode, 0x20),
            mload(creationCode)
        )
    }

    if (addr == address(0)) {
        revert TokenFactory__DeploymentFailed();
    }

    s_tokenToAddress[symbol] = addr;
    emit TokenDeployed(symbol, addr);
}
```

For ZKsync, use the supported deployment method described in H-4 rather than arbitrary raw bytecode.

Medium

[M-1] Unbounded return data from arbitrary targets can cause gas griefing during withdrawals

Description

`sendToL1` performs a low-level call to an arbitrary target and forwards nearly all available gas:

```
(bool success,) =
    target.call{value: value}(data);
```

A malicious target can return or revert with an excessively large amount of return data. Standard Solidity low-level calls may copy that data into the caller's memory, causing expensive memory expansion and return-data copying.

This technique is commonly called a **return bomb**.

The issue is most relevant when a bridge-operated relayer or another third party pays to execute signer-approved withdrawal messages. If users only execute their own messages, the attacker mainly griefs their own transaction.

Impact

A malicious target may:

- Cause the withdrawal transaction to consume unexpectedly large amounts of gas.
- Make execution revert with an out-of-gas error.
- Increase relayer operating costs.
- Disrupt automated withdrawal processing.
- Force repeated execution attempts.

The attacker cannot spend more than the transaction's configured gas limit, but a poorly configured relayer can be induced to waste substantially more gas than expected.

Proof of Concept

An adversarial target can return a very large data region:

```
contract ReturnBomb {
    fallback() external payable {
        assembly {
            return(0, 1_000_000)
```

```
        }
    }
}
```

A signer-approved message targeting this contract causes `sendToL1` to execute the malicious fallback:

```
bytes memory message = abi.encode(
    address(returnBomb),
    0,
    bytes("")
);
```

The exact gas consumed depends on compiler settings and the transaction gas limit, but the target controls the size of the return data that the caller may attempt to copy.

Recommended Mitigation

When return data is not required, use a custom assembly call that copies zero return bytes and forwards a bounded amount of gas:

```
function _safeCall(
    address target,
    uint256 value,
    bytes memory data,
    uint256 gasLimit
) internal returns (bool success) {
    assembly {
        success := call(
            gasLimit,
            target,
            value,
            add(data, 0x20),
            mload(data),
            0,
            0
        )
    }
}
```

Alternatively, use a reviewed library such as Nomad's `ExcessivelySafeCall` and cap the maximum copied return-data length:

- https://github.com/nomad-xyz/ExcessivelySafeCall

This mitigation should be combined with H-3: ordinary token withdrawals should not be able to target arbitrary contracts in the first place.

Low

[L-1] Critical signer and withdrawal actions are not consistently emitted as events

Description

`setSigner` modifies the trusted signer set without emitting a protocol-specific event.

Successful calls to `sendToL1` and `withdrawTokensToL1` also do not emit a withdrawal-execution event.

Although OpenZeppelin's `Pausable` contract emits events for pause-state changes, the bridge lacks equivalent observability for signer changes and withdrawals.

Impact

Off-chain monitoring systems cannot reliably:

- Track signer additions and removals.
- Reconcile executed withdrawals.
- Detect unusual withdrawal destinations or amounts.
- Alert on signer-key compromise.
- Build a complete protocol accounting history from events.

Recommended Mitigation

Emit events containing sufficient indexed data:

```
event SignerUpdated(
    address indexed signer,
    bool enabled
);

event WithdrawalExecuted(
    bytes32 indexed withdrawalId,
    address indexed signer,
    address indexed target,
    uint256 value,
    bytes32 dataHash
);
```

```
function setSigner(
    address account,
    bool enabled
) external onlyOwner {
    signers[account] = enabled;
    emit SignerUpdated(account, enabled);
}
```

Emit the withdrawal event only after successful execution.

[L-2] Duplicate token symbols overwrite previous factory deployments

Description

`TokenFactory` stores one token address per string symbol:

```
mapping(string tokenSymbol => address tokenAddress)
    private s_tokenToAddress;
```

`deployToken` does not check whether a symbol has already been registered:

```
s_tokenToAddress[symbol] = addr;
```

Deploying another token with the same symbol silently overwrites the previous mapping entry.

Impact

The original token remains deployed but becomes unreachable through `getTokenAddressFromSymbol`.

Users, deployment scripts, indexers, or bridge components may resolve the symbol to an unexpected token address. This creates operational ambiguity and can lead to integration errors.

Proof of Concept

```
function testDuplicateSymbolOverwritesTokenAddress()
    public
{
    TokenFactory factory = new TokenFactory();

    address first = factory.deployToken(
```

```
        "BBT",
        type(L1Token).creationCode
    );

    address second = factory.deployToken(
        "BBT",
        type(L1Token).creationCode
    );

    assertTrue(first != second);
    assertEq(
        factory.getTokenAddressFromSymbol("BBT"),
        second
    );
}
```

Recommended Mitigation

Reject duplicate symbols:

```
error TokenFactory__SymbolAlreadyRegistered();

function deployToken(
    string calldata symbol,
    address recipient
) external onlyOwner returns (address addr) {
    if (
        s_tokenToAddress[symbol] != address(0)
    ) {
        revert TokenFactory__SymbolAlreadyRegistered();
    }

    addr = address(new L1Token(recipient));
    s_tokenToAddress[symbol] = addr;

    emit TokenDeployed(symbol, addr);
}
```

If multiple deployments per symbol are intentional, store an array or use a unique token identifier rather than silently overwriting the previous address.

[L-3] Compiler-generated `PUSH0` bytecode may be incompatible with the intended L2 configuration

Description

Solidity `0.8.20` targets the Shanghai EVM by default and may emit the `PUSH0` opcode introduced by EIP-3855.

At the time and configuration targeted by the reviewed codebase, the intended ZKsync Era deployment path did not necessarily support Ethereum Shanghai bytecode in the same manner as Ethereum Mainnet.

This is a deployment-compatibility risk rather than an application-logic vulnerability.

Impact

Contracts compiled with incompatible bytecode settings may fail to deploy or execute on the intended L2 environment.

Recommended Mitigation

Test deployment using the exact production compiler and ZKsync tooling.

Where required, compile Ethereum bytecode against a pre-Shanghai EVM target:

```
# foundry.toml
evm_version = "paris"
```

For native ZKsync deployment, use the supported `zksolc`/Foundry-ZKsync workflow and verify all generated bytecode against the target network before production deployment.

Because ZKsync compatibility evolves, this finding must be revalidated against the exact production VM mode and network release.

Informational

[I-1] Security-critical paths require stronger negative and invariant test coverage

Description

The protocol's security depends on cross-contract and off-chain invariants that are not adequately represented by basic happy-path unit tests.

Important properties include:

```
Total L1 withdrawals must never exceed authenticated L2 burns.
Each withdrawal authorization must be executable at most once.
Only the token owner may initiate an L1 deposit.
Each Deposit event must correspond to a real vault balance increase.
Unsolicited vault transfers must not consume bridge deposit capacity.
Signer-approved messages must not execute arbitrary privileged calls.
```

Recommended Mitigation

Add regression tests for every finding in this report and maintain stateful invariants such as:

```
function invariant_withdrawalsAreFullyAuthorized()
    public
    view
{
    assertLe(
        handler.totalWithdrawn(),
        handler.totalAuthenticatedBurns()
    );
}
```

Recommended coverage areas:

- Replay attempts
- Duplicate withdrawals
- Arbitrary `from` addresses
- Vault-to-vault transfers
- Direct token donations to the vault
- Malicious return-data targets
- Duplicate factory symbols
- Failed deployments
- Signer rotation
- Pause/unpause behavior

- Cross-chain message domain separation

Aim for high branch and invariant coverage, not only a high aggregate line-coverage percentage.